Strings & Bioinformatic files

Contents

Why Strings?

Why Strings? I

- Strings or texts play an essential role in bioinformatics, be it in form of sequences (RNA / DNA or protein) or in form of text files, that contain biological / biochemical / bioinformatical data
- Therefore it‘s important to know how deal with strings and what kind of methods are available to you
- When working with files like fasta, genbank and so on, we are mostly interested in extracting information from them, to use these information later on in our calculations

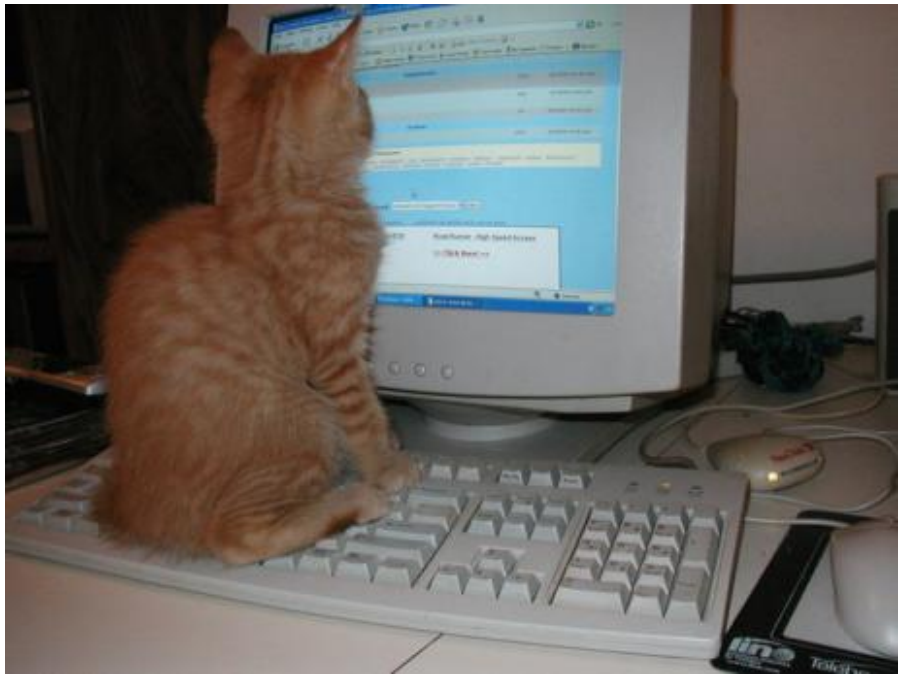
Why Strings? II

- When working with sequences however we want to search for patterns that allow us to make guesses about our new sequences in relation to function, phylogenetics etc.
- Therefore we must compare new sequences with already known sequences
- The methods we use here are database searches using algorithms like BLAST, comparing multiple sequences at once using Multiple sequence alignment and comparing two sequences in depth using pairwise Alignments or Dotplots

Strings in informatics

Strings in informatics

- When looking at strings in informatics we only see a collection of letters and don‘t interpret the meaning behind the letter combination
- That means all algorithms we will talk about do not care for the biological meaning of a mutation for example
- So then comparing two sequences they just see that some letters are different and nothing more



Definitions I

- Alphabets
  - Σdna = {A,G,C,T} ; for nucleotides
  - Σamino = {A,C,D,E,F,G,H,I,K,L,M,N,P,Q,R,S,T,V,W,Y}; for amino acids
- Each **Element E** of an alphabet is named symbol or letter
- **String S** is a juxtaposition of symbols or letters from the alphabet Sigma
  - S = (ΣE) (An order of symbols is defined)
  - E.g: S = „ATGGAATGCTAATAG“

Definitions II

- **Position Si** is the letter E on position i in the string (starts at position 0!)
- **Substring Sij** are the letters from position Si to Sj excluding the latter
- **Length N** of a string is the number of letters in it
- **Split Spliti** is the splitting of a string in two substrings at position i. Two new strings S0i and SiN are created
- **Split SplitDelimitor** is the splitting of a string in multiple substrings divided at the delimitor. The number of substrings is equal to the number of occurrences of the delimitor +1
- **Prefix n** is a substring of the string S which contains the first n letters of S
- **Suffix n** is a substring of the string S which contains the last n letters of S

Position Si I

- In Python we can access a letter at Position i in our string S by using indices, similar to lists
- For the String S down below the position S0, S1 and S2 could be accessed with their corresponding index:

```
S = "Examplestring"

S[0] = "E"

S[1] = "x"

S[2] = "a"
```

- The last possible index for this example would be 12 so using any bigger number as index would result in an error

Position Si II

- When working with lists, we learned that negative indices allow us to access values from the end of our lists
- The same holds true for strings, so we can access the last letter of our string S with the index -1:

```
S = "Examplestring"

S[-1] = "g"
```

Substring Sij I

- In python we can generate substrings by splicing our string S again with the help of indices
- We discussed this principle behind slicing when we were talking about lists and it follows the same principles here
- We create a substring Sij with i being the Start_index and j being the End_index by using the following principle:

    S[Start_index(included):End_index(excluded)]

Substring Sij II

- So for example we can create the substrings S25, S05 and S213 as follows:

```
S = "Examplestring"

S25 = S[2:5]        ➔   "amp"

S05 = S[:5]         ➔   "Examp"  (Could be written as S[0:5])

S213 = S[2:]        ➔   "amplestring"  (or S[2:13])
```

- As you can see for substrings that start with the index 0 we can leave this index out and just write :n instead of 0:n
- On the other hand for substrings that go on until the end of our string S we can leave out the End_index and just write 2: instead of 2:13

Length N

- The length N of a string represents the numbers of letters / symbols in it
- In python we can use the built-in function len() to determine this parameter of our string:

```
S = "Examplestring"

lengthN = len(S)
```

➔ lengthN = 13

- When using the function len() we need to keep in mind that all kinds of symbols are counted
- That means spaces in our string, special characters like "!" as well as escape characters like a line-break are seen as part of our string and therefore are also counted

Exercise 01: Simple Strings

- Write a Python script that defines five string variables containing these Strings:
  - “AAAAAAAAAAAAAAAAA”
  - “ACTGACTGACTGACTGACTG”
  - “HAIMGVVFTWIMALACAAPPLVGWSRY”
  - “SSSIYNPVIYIMLNKQFRNCMLTTLCCGKNPLG”
  - “PFSNVTGVVRSPFEQPQYYLAEPWQFSMLAAYMFLLIVLGFPINFLTLYVTVQH”
- Print out the length of all five variables.
- Print out the letters at the indices 0,5,10,15,20 and 25 of the five strings if possible
- Create the substrings S-5-20, S-20-30, S-2-10 of all five strings

String methods

What are String methods

- All functions in python that directly or indirectly work with strings are called string methods
- These can range from simple things like splitting up a String over searching for Substrings up until replacing substrings
- During this course we will discuss some of the major string methods you need when working with Python

Split SplitDelimitor I

- We saw earlier how to create substrings using indices
- Sometimes we need to split up our strings at each occurrence of a specific symbol or a specific combination of letters
- In these cases we can use the built-in method split() to do so
- This method is similar to list methods like append() or extend() and has to be used with the help of the point notation and the result is a list with our substrings:

```
S = "This is a test"

list1 = S.split()
```

➔ list1 = [“This”,”is”,”a”,”test”]

- As you can see split() works without any arguments and in this case it uses spaces as delimitors

Split SplitDelimitor II

- Please keep in mind that the delimitor you use when working with split() is removed from the resulting list
- The general principle behind split() is as follows:
  - First it opens a substring
  - Than the function moves through our string symbol by symbol and as long as these symbols do NOT match the delimitor they are added to the open string
  - Once it encounters the delimitor the current substring is closed, the delimitor is removed and a new substring is opened
  - Afterwards the function continues like this until it reaches the last symbol of our string and closes the last substring

Split SplitDelimitor III

- We can use split with one argument as well and than this argument, which has to be a string is used as delimitor
- If the handed over delimitor is not part of our string we will still get back a list but now this list contains only one element, our whole string S

```
list2 = S.split(",")
```

➔ list2 = [“This is a test”]

- We can only use ONE delimitor with split() so its functionality is a bit limited here, but we will talk about other methods later

find I

- We can search for substrings in our string S with the help of the method find, that again uses the point notation:

```
S = "This is a test"
F1 = S.find("s")
```

➔ F1 = 3

- As you can see find returns an integer value that represents the start index of the first occurrence of the search term, here “s”
- Of course our search term could be longer as well:

```
F2 = S.find("test")
```

➔ F2 = 10

- The value we get back in this case is still just one number and represent the index of the first letter, here “t”, of our search term, here “test”

find II

- When working with methods like find, that search for a specific substring for example we need to keep in mind case sensitivity
- That means for these methods “a” is different from “A”, unlike SQL for example
- We can use find even with search terms that are not part of our string without producing any errors:

```
F3 = S.find(“this“)
```

➔ F3 = -1

- However in these cases find returns -1
- It was implemented like this to allow a differentiation between a hit at the last letter of our string and no hit at all

Using in

- Sometimes you only want to know if a substring is in your string and don’t care about the position where to find it
- In these cases the keyword in can be used:

```
S3 = " A third example for a String!“
print(„third“ in S3)
```

➔ prints True

```
if „third“ in S3:
    print(„You found your word!“)
```

➔ returns True

➔ is printed

Exercise 02: String methods

- Use the strings from Exercise 01 to fulfill the following tasks:
  - At which index are the first occurrences of the letters A, C, G, N, T and U
  - Split the strings at each appearance of the patterns “TG”, “TT” and “AA”
  - Use if statements to test whether the strings contain the patterns “FLL” and “MLT”

What is the module re [1]

- re stands for Regular expression operations
- It‘s a strong built-in module in Python that allows us to work with strings in a much more complex way
- It contains several very useful string methods you should know, which can come in handy later on
- I will show you 3 methods I personally like to use
- To use this module we have to import it before we are able to use its functions later on:
  - `import re`
- After the import we can use the functions of re with the point notation

re.split() I

- The first method is re.split() which similar to the built-in method split() allows us to create Split SplitDelimitors of our String
- The main advantage here is that we can hand over multiple delimitors separated by pipes as one argument of it
- I takes two arguments first the delimitors and second the string it should split according to these delimitors:

```
S3 = " A third example for a String, this time with punctuation!“
Sp9 = re.split(" |,|;|!",S3)
```

➔ Sp9 = ['', 'A', 'third', 'example', 'for', 'a', 'String', '', 'this', 'time', 'with', 'punctuation', '']

re.split() II

- Aside of using multiple delimitors at once re.split() works nearly in the same way as the built-in split
- However there is one essential difference in the resulting list concearning delimitors at the end or start of our string as well as multiple delimitors directly after each other (two spaces for example):
  - The built-in method split() ignores the resulting empty string it would produce
  - re.split() includes these empty strings in the resulting list

re.split() III

- Imagine a string S5 as shown below
- It has a space at the start and the end of it as well as two spaces directly after each other between the two words:

```
S5 = " Hi  You "
```

- If we now use the built-in method split() as well as re.split() on it with the delimitor of a space we will get very different results:

```
list4 = S5.split()          ➔ list4 = ['Hi','You']
list5 = re.split(' ',S5)    ➔ list5 = ['','Hi','','You','']
```

- As you can see re.split() produced three additional empty strings as elements in the resulting list

Insert: Wildcard in the module re

- The module re allows the usage of one wildcard in it‘s functions
- This wildcard is the period sign “.”
- So if you want to use a “.” as delimitor or so you need to escape it in the corresponding function
- That means you need to use “\\.” instead of just “.”

re.finditer() I

- The second method is re.finditer() which allows us to find the start index off all occurences of a subtring inside our String S3
- Unfortunately re.finditer() returns a generator which contains so called match objects, one for each occurrence of our substring
- To access the start index of each match object we have use the function start() on each of these objects

re.finditer() II

- The “easiest” and in my opinion most convenient way of getting a list with the corresponding start indices is by using a list comprehension

```
S3 = " A third example for a String, this time with punctuation!“
I3 = [m.start() for m in re.finditer("a",S3)]
```

➔ I3 = [11, 21, 52]

- Let‘s go over this construct step by step
- At the center we have the re.finditer() function that takes two arguments, first the substring we are searching for (here “a”) and second the string we are searching in (here S3):

```
re.finditer("a",S3)
```

re.finditer() III

- Than we have a for loop that allows us to iterate over the result of the call back of the re.finditer() function:

```
for m in re.finditer("a",S3)
```

- Our loop variable m takes on the values of the different match objects in the result of re.finditer()
- Than we tell Python what we want to do with these match objects m in each iteration, here we invoke the start() function on them:

```
m.start() for m in re.finditer("a",S3)
```

re.finditer() IV

- By enclosing this whole construct in square brackets we create a list that contains the result of the whole process:

```
[m.start() for m in re.finditer("a",S3)]
```

- That means we iterate over the result of re.finditer() read out the start index of each match object and place the corresponding values one after the other into a list

re.finditer() V

- We could generate the same result with the help of a normal for loop as well:

```
I3 = []
for m in re.finditer("a",S3):
        I3.append(m.start())
```

- However as you can see we now need at least three lines of code instead of a single line
- To be honest nearly everyone simply copies this construct and just adjusts the parameters of re.finditer to his or her needs without trying to figure out what this construct does in detail

re.findall()

- The last method is re.findall() that similar to re.finditer() allows us to find how often a specific substring appears in our string
- However re.findall() returns a list that contains the substring we are searching for n-times, with n being the number of occurences of this substring:

```
Sf = re.findall("a",S3)
```
➔ Sf = ["a","a","a"] ➔ len(Sf) = 3

- In most cases the list we get back here is not really useful, so therefore I use findall() mainly in combination with len() to directly determine how often my substring appeared in the string S3:

```
Sfn = len(re.findall("a",S3))
```
➔ Sfn = 3

Exercise 03: Working with Strings

- Take the “lorem ipsum” String from down below and use it as a variable in Python
- Complete the following tasks:
  - How long is this text?
  - How many words are in this text? (split and len)
  - How often does each letter of the alphabet occur in this text? (re, len and a loop)
  - Are the words “minim”, “anim”, “caesar”, “brutus” and “duis” in the string?

Exercise 03: Lorem Ipsum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Modifying strings I

- Up until now, even though we splitted our Strings, we didn‘t make any changes to our letters / symbols inside our strings
- But in many cases we need to make some changes to our letters
- For example setting everything to lower or upper case, to make sure our if statements work correctly
- There are in total four different methods I want to show you, which can be used to make these changes

Modifying strings II

- The first method is called upper() and returns the string it's invoked on in all upper case:

```
S = "This is a Test"
Supper = S.upper()
```

➔ Supper = "THIS IS A TEST"

- The second method lower() returns the string it's invoked on in all lower case:

```
S = "This is a test"
Slower = S.lower()
```

➔ Slower = "this is a test"

Modifying strings III

- The third method strip() removes whitespaces at the beginning or the end of the string

```
S = " This is a Test "

Sstrip = S.strip()
```

➔ Sstrip = “This is a Test”

- Last but not least replace() can be used to change one substring into another
- Unlike the other three this method takes two arguments, first the substring you want to replace and second the replacement string:

```
S = "This is a Test"

Sreplace = S.replace("s","W")
```

➔ Sreplace = “ThiW iW a TeWt”

Modifying strings IV

- Even though all four methods use the point notation the changes we create here are NOT permanent
- That means our string S is still the same and we need to assign the result to a new variable for example:

```
Snew = S.upper()
```

- However we can of course use a Re-Assignment to make these changes permanent:

```
S = S.upper()
```

Concatenating Strings

- We can combine strings simply by using a „+“ operator to concatenate them:

```
S1 = "This string is an example"
S2 = "Another example for a long string„
S4 = S1 + S2
```

➔ S4 = "This string is an exampleAnother example for a long string"

- As you can see the „+“ added together our strings just fine, BUT we don‘t have a space between them because the first letter of S2 (“A”) is placed directly after the last letter of S1 (“e”)

Exercise 04 – A bit more about Strings

- Take the “lorem ipsum” string from Exercise 03 and complete the following tasks:
  - Replace the strings et, in and eu with at, on and au
  - Create an all lower case and an all upper case variant of it
  - Split the text in it’s words and concatenate them together again (split and a loop)

Working with Files

Writing Files I

- Python allows us to write and read in text files in a relative simple manner
- When working with files we always need the open() function that allows us to open a (non-)existing file and work with its contents
- open() takes two arguments, first the name of the file you want to work with and second an operator that tells Python what you want to do with this file
- There are three operators available:
  1. “w” for writing, overwrites already existing contents of your file
  2. “a” for appending, adds text to the end of your file
  3. “r” for reading, works only in combination with already existing files

Writing Files II

- Let‘s say we want to open a file to write something in it, in this case we could use the open() function as follows:

```
open("TestFile.txt","w")
```

- The open function itself, is similar to the MySQLConnection object we talked about during the SQL in Python part, that means it just connects our Python script to the file
- Thus we must name this object:

```
file = open("TestFile.txt","w")
```

Writing Files III

- Once you opened your file you‘re now able to add texts to your file with the help of the write() function of the file object we just created:

```
file.write()
```

- This function takes only strings as arguments and as the name implies writes them into our file:

```
file.write(“This is a test”)
```

- Unfortunately this function does not create any line breaks, therefore we have to add them on our own by using the escape character “\n”:

```
file.write(“This is a test\n”)
```

Writing Files IV

- We can also write the values of variables into our files by including their name as argument for the write() function:

```
string1 = "Hello Christoph„
file.write(string1)
```

- However keep in mind that you have to cast these variables to string if they‘re NOT of the datatype string:

```
float1 = 456.34
file.write(str(float1))
```

Writing Files V

- Even though write() takes only one argument we can use the “+” operator inside it to concatenate multiple strings, which are than seen as only one argument to write:

```
file.write(“This is a test “+string1+”\n”)
```

- Last but not least we could generate a f-string and use this f-string as argument for write():

```
string2 = f"This is a test {string1} you earn {float1}"
file.write(string2)
```

Writing Files VI

- After you wrote everything into your file you have to close it, to save all changes you made to your file
- This is done with the close() function of our file object:
  - `file.close()`
- When you want to write something into a file, the corresponding file does NOT need to exist
- That means if you specify a non-existing file as the first argument for open, as long as you use the operator “w”, the file is simply created on the spot
- HOWEVER this only works for “w” or “a”, if you use “r” the file NEEDS to exist

Writing Files VII

- Instead of creating a file object in form of a variable declaration we can work on Files with the help of the keyword with
- Similar to what we saw when talking about biopython we are able to create a with statement as follows:

```
with open("TestFile2.txt","w") as with_file:
        with_file.write("This is a test\n")
```

- As long as we are inside our with statement we can write the contents to our file
- The big advantage of using a with statement is that we don‘t need to close our file and thus all changes are saved as soon as we leave the with statement later on or the script is finished executing

Exercise 05 – Writing Files

- Write a program that creates two different files.
- The first file should be opened normally and two lines should be written in it.
- Afterwards close the file, open it again and add another three lines of text to the file (use the operator „a“ for append)
- The second file should be generated by using „with“ and a loop to write a countdown from 10 to 0
- Each number should be in a separate line

Reading Files I

- When reading in files you should always use a with statement to avoid leaving your file open after you‘re finished reading in the contents:

```
with open("TestFile.txt","r") as read_file:
```

- Inside the with statement you can now use three different ways to read in the contents of your file

Reading Files II

- The first one is by using the read() function of the file object you created in your with statement:

```
with open("TestFile.txt","r") as read_file:
    content = read_file.read()
```

- The variable content now contains the whole text of your file as one big string, including any line breaks, special characters and so on
- Please use this version only if your file is very small, since you put the whole file into your memory at once

Reading Files III

- Another version is to use the function readlines() of your file object to create a list of the contents of your file:

```
with open("TestFile.txt","r") as read_file2:
    lines = read_file2.readlines()
```

- The variable lines is now a list and each element of this list is a string containing the text of one line of your file
- This should also only be used with small files because you again read in the whole file into the memory

Reading Files IV

- Last but not least we can loop over our file line by line inside the with statement to read in each line separately:

```
with open("TestFile.txt","r") as read_file3:
    for line in read_file3:
        print(line)
```

- This is the most common and also best method when reading in files, since we only have one line of text in our memory at all times and we are able to modify each line separately to read out only the most important data from it

Exercise 06 – Reading a File

- Read in your second file from exercise 5, the one with the countdown, by using the last method we discussed, the one using a for loop inside the with statement
- Calculate the sum of the numbers and print out the result

Parsing Files I

- Inputs and outputs of bioinformatic programs are often from text files or similar things
- Not all information contained in these files are always necessary for the handling of the problem we are tackling
- You want to pick only the information you really need and this process is called parsing
- Parsing requires the file to follow a specific format and you need to know the specifications of this format

Parsing Files II

- When parsing a file we ALWAYS have to follow these 4 general steps:

1. Read the file
2. Iterate over the file line by line
3. Disassemble the lines according to format so that only the necessary information is retained – Know the format
4. Save the data you need in a Python-usable datastructure of your choice

Parsing Files III

- Before you start writing a parser ask yourself the following questions:

1. Can I use an already existing parser?
2. Do I need to make the parser by myself?
3. Which information do I need?
4. How do I have to dismantle each line?
5. Which substrings can be used as marker for my data?

Bioinformatic file types

Bioinformatic file types I

- In bioinformatics you will work with lots of different file types
- Many of these types are simple text files that follow specific formatting rules and have specific extensions instead of .txt
- Knowledge about the format of these files as well as which extensions you can use to make sure that your programs can read in these files is important
- However we can‘t tackle all of these types here, since just for sequences there are more than 31 different file types available to us
- That‘s why we will focus on the most common ones

Bioinformatic file types II

- The most common ones are:
  - fasta
  - fastq
  - genbank
  - clustal
  - pdb
  - xml
  - json
  - mmCIF
  - …
- On the next slides we will discuss the format of these files and which parsers are available to you in Python
- We will also look at the objects these parsers produce so that you can get an idea which parser fits your needs the most
- At the end of the day you need to decide which parser and maybe even which type of file you want / need to work with

fasta files I

- Fasta files are used to save and relay raw sequences with only a small amount of additional data
- They can have the following extensions:
  - .fasta, for general sequences
  - .faa, for protein sequences
  - .fna, for nucleic acid sequences
- The different extensions do NOT influence the format of these files, they simply help us to identify the type sequence(s) our file contains

fasta files II

- These files are formatted very simple since a line can either contain the name of a sequence or the sequence itself
- Lines that contain the name of a sequence start with “>” and they can span only exactly one line
- All lines afterwards that don’t start with “>” than contain the sequence corresponding to the name above
- That means one sequence can span multiple lines

fasta files III


```
>sp|P19367|HXK1_HUMAN Hexokinase-1 OS=Homo sapiens GN=HK1 PE=1 SV=3
MIAAQLLAYYFTELKDDQVKKIDKYLYAMRLSDETLIDIMTRFRKEMKNGLSRDFNPTAT
VKMLPTFVRSIPDGSEKGDFIALDLGGSSFRILRVQVNHEKNQNVHMESEVYDTPENIVH
GSGSQLFDHVAECLGDFMEKRKIKDKKLPVGFTFSFPCQQSKIDEAILITWTKRFKASGV
EGADVVKLLNKAIKKRGDYDANIVAVVNDTVGTMMTCGYDDQHCEVGLIIGTGTNACYME
ELRHIDLVEGDEGRMCINTEWGAFGDDGSLEDIRTEFDREIDRGSLNPGKQLFEKMVSGM
YLGELVRLILVKMAKEGLLFEGRITPELLTRGKFNTSDVSAIEKNKEGLHNAKEILTRLG
VEPSDDDCVSVQHVCTIVSFRSANLVAATLGAILNRLRDNKGTPRLRTTVGVDGSLYKTH
PQYSRRFHKTLRRLVPDSDVRFLLSESGSGKGAAMVTAVAYRLAEQHRQIEETLAHFHLT
KDMLLEVKKRMRAEMELGLRKQTHNNAVVKMLPSFVRRTPDGTENGDFLALDLGGTNFRV
LLVKIRSGKKRTVEMHNKIYAIPIEIMQGTGEELFDHIVSCISDFLDYMGIKGPRMPLGF
TFSFPCQQTSLDAGILITWTKGFKATDCVGHDVVTLLRDAIKRREEFDLDVVAVVNDTVG
TMMTCAYEEPTCEVGLIVGTGSNACYMEEMKNVEMVEGDQGQMCINMEWGAFGDNGCLDD
IRTHYDRLVDEYSLNAGKQRYEKMISGMYLGEIVRNILIDFTKKGFLFRGQISETLKTRG
IFETKFLSQIESDRLALLQVRAILQQLGLNSTCDDSILVKTVCGVVSRRAAQLCGAGMAA
VVDKIRENRGLDRLNVTVGVDGTLYKLHPHFSRIMHQTVKELSPKCNVSFLLSEDGSGKG
AALITAVGVRLRTEASS
```

- As you can see the name line can contain additional information as well, depending on the source of your file (here UniProt)

fasta files IV

- Fasta files can contain any number of sequences, simple separated by their corresponding name lines
- In the example on the right side, we see three different protein sequences that are saved in the same file

```
>NP_663358.1 rhodopsin [Mus musculus]
MNGTEGPNFYVPFSNVTGVVRSPFEQPQYYLAEPWQFSMLAAYMFLLIVLGFPINFLTLYVTVQHKKLRTPLNYILLNLA
VADLFMVFGGFTTTLYTSLHGYFVFGPTGCNLEGFFATLGGEIALWSLVVLAIERYVVVCKPMSNFRFGENHAIMGVVFT
WIMALACAAPPLVGWSRYIPEGMQCSCGIDYYTLKPEVNNESFVIYMFVVHFTIPMIVIFFCYGQLVFTVKEAAAQQQES
ATTQKAEKEVTRMVIIMVIFFLICWLPYASVAFYIFTHQGSNFGPIFMTLPAFFAKSSSIYNPVIYIMLNKQFRNCMLTT
LCCGKNPLGDDDASATASKTETSQVAPA
>XP_031236805.1 rhodopsin [Mastomys coucha]
MNGTEGPNFYVPFSNITGVVRSPFEQPQYYLAEPWQFSMLAAYMFLLIVLGFPINFLTLYVTVQHKKLRTPLNYILLNLA
VADLFMVFGGFTTTLYTSLHGYFVFGPTGCNLEGFFATLGGEIALWSLVVLAIERYVVVCKPMSNFRFGENHAIMGVAFT
WVMALACAAPPLVGWSRYIPEGMQCSCGIDYYTLKPEVNNESFVIYMFVVHFTIPMIVIFFCYGQLVFTVKEAAAQQQES
ATTQKAEKEVTRMVIIMVIFFLICWLPYASVAMYIFTHQGSNFGPIFMTLPAFFAKSSSIYNPVIYIMLNKQFRNCMLTT
LCCGKNPLGDDDASATASKTETSQVAPA
>AAA39861.1 opsin [Mus musculus]
MNGTEGPNFYVPFSNVTGVGRSPFEQPQYYLAEPWQFSMLAAYMFLLIVLGFPINFLTLYVTVQHKKLRTPLNYILLNLA
VADLFMVFGGFTTTLYTSLHGYFVFGPTGCNLEGFFATLGGEIALWSLVVLAIERYVVVCKPMSNFRFGENHAIMGVVFT
WIMALACAAPPLVGWSRYIPEGMQCSCGIDYYTLKPEVNNESFVIYMFVVHFTIPMIVIFFCYGQLVFTVKEAAAQQQES
ATTQKAEKEVTRMVIIMVIFFLICWLPYASVAFYIFTHQGSNFGPIFMTLPAFFAKSSSIYNPVIYIMLNKQFRNCMLTT
LCCGKNPLGDDDASATASKTETSQVAPA
```

fasta files V

- When working with fasta files we have at least two already existing parser we can use, aside of writing our own parser
- The first one is included in the SeqIO submodule of biopython [2]
- As we know we can import this submodule as follows:

```
from Bio import SeqIO
```

- Afterwards we are able to parse fasta files with the parse() function as follows:

```
fasta1 = SeqIO.parse("B3JI28.fasta", "fasta")
```

- Be aware that we are using the parse() function instead of the read() function, since this function allows us to work with files that contain multiple sequences

fasta files VI

- When using the parse() function of SeqIO we generate a FastaIterator object, that contains multiple sequence objects, one for each sequence in our file
- We can iterate over these sequence objects with the help of a for loop for example:

```
for record in fasta1:
```

- Inside the loop we can now access for example the sequence by using the attribute seq of the sequence object record:

```
print(record.seq)
```

fasta files VII

- The second method is to use the module fastaparser [3] which also allows us to read in fasta files
- We get access to this module by importing it as follows:

```
import fastaparser
```

- Afterwards we need a with statement to open the fasta file so that fastaparser can read it in:

```
with open("B3JI28.fasta","r") as fasta_file:
```

fasta files VIII

- Inside our with statement we are now able to read in our file with the help of the Reader() function of fastaparser
- We can either use the default options to generate a FastaSequence object:

```
fasta1 = fastaparser.Reader(fasta_file)
```

- Or we can use the so called quick method that saves time and reads in only the bare minimum of Header and sequence from our fasta file in form of named tuples:

```
fasta1 = fastaparser.Reader(fasta_file, parse_method='quick')
```

fasta files IX

- Afterwards, still inside the with statement, we can iterate over the object we created with the Reader() function by using a simple for loop:

```
for seq in fasta1:
```

- Depending on the method we used earlier with the Reader() function we can access our sequences differently
- If we created a FastaSequence object by using the default options we need the function sequence_as_string() from our object inside the loop:

```
print(seq.sequence_as_string())
```

- If we used the quick method we can just use the attribute sequence:

```
print(seq.sequence)
```

fasta files X

- Last but not least you could write your own parser, for example to generate a dictionary that contains your sequences later on
- The script “16_Files-Fasta-Own-Parser.py” contains a simple example how to write such a parser with the knowledge you have until this point

Exercise 07: Small sequence comparison

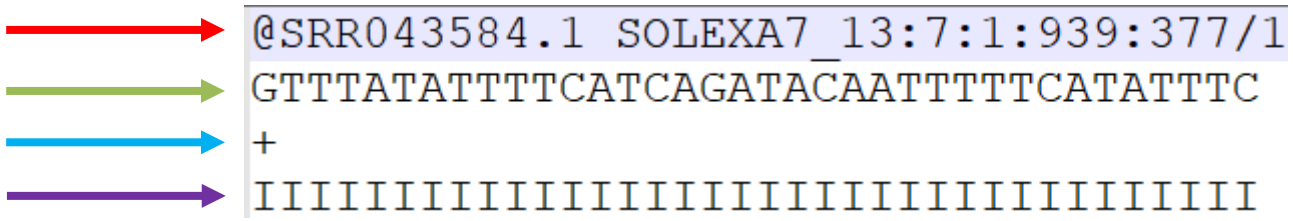
- You have four fasta files called XP_021046627, XP_031236805, A0A8I4A5I4 and ELR51227
- Write a program that reads in all four of them
- Afterwards compare the first three sequences letter by letter with each other (use a simple loop) and calculate how many identical letters these sequences have
- Compare the fourth sequence with the remaining three in a similar matter, be careful this sequence is 3 amino acids longer

fastq files I [9]

- Format for storing both a biological sequence (usually nucleotide sequence) and its corresponding quality scores
- Both the sequence letter and quality score are each encoded with a single character for brevity
- Originally developed at the Wellcome Trust Sanger Institute to bundle a FASTA formatted sequence and its quality data
- Has recently become the de facto standard for storing the output of high-throughput sequencing instruments such as the Illumina Genome Analyzer

fastq files II [9]

- A FASTQ file has four line-separated fields per sequence:
  - Field 1 begins with a '@' character and is followed by a sequence identifier and an optional description (like a FASTA title line) (red arrow)
  - Field 2 is the raw sequence letters (green arrow)
  - Field 3 begins with a '+' character and is optionally followed by the same sequence identifier (and any description) again (blue arrow)
  - Field 4 encodes the quality values for the sequence in Field 2, and must contain the same number of symbols as letters in the sequence (purple arrow)



fastq files III [9]

- The byte representing quality runs from:
  - 0x21 ➔ lowest quality ➔ '!' in ASCII
  - 0x7e ➔ highest quality ➔ '~' in ASCII
- The quality value characters in left-to-right (two rows) increasing order of quality (ASCII) are:

```
!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNO
PQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
```

fastq files IV [9]

- Original FASTQ files split long sequences and quality strings over multiple lines, as is typically done for FASTA files
- This makes parsing more complicated due to the choice of "@" and "+" as markers (as these characters can also occur in the quality string)
- Multi-line FASTQ files (and consequently multi-line FASTQ parsers) are less common now that the majority of sequencing carried out is short-read Illumina sequencing, with typical sequence lengths of around 100bp

fastq files V [9]

- fastq files can of course contain multiple snippets of different sequences as it‘s shown on the right side
- Each sequence consists of it‘s own four lines representing the different fields

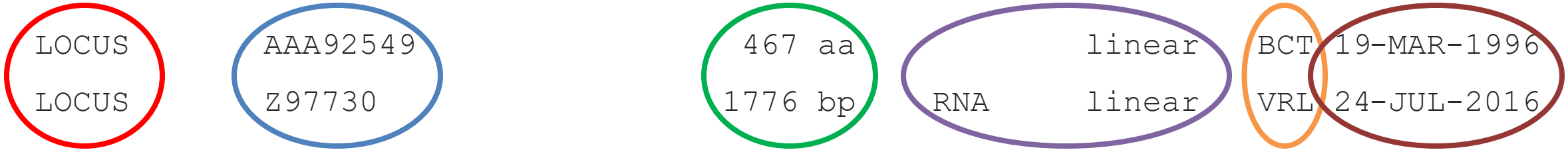
```
@SRR043584.9441520 SOLEXA7_13:7:100:1469:550/1
TCGATGAGTCGACGAAGATCGGAAGAGCTCGTATGC
+
93I/>6/5).?1,<-),+5(B,G+6.HG8)0.,2,&
@SRR043584.9441521 SOLEXA7_13:7:100:1749:1249/1
GGTCTCATTCTGTAGAACTGGTTGGTCAGGAAGACA
+
;I7/BIII(*ICAIII6+III&IA7+#II1I'?.II
@SRR043584.9441522 SOLEXA7_13:7:100:1199:767/1
TCTGCTACTTTTCCTTATTGCGTTTGATTTTGATTT
+
++I.I&ID+I2)*2$=4D@II2IIIII6%67I?;,I
```

genbank files I

- genbank files contain more information than simple fasta files and are mainly used by the NCBI and its databases Nucleotide or Gene
- They often have the extension .gb
- In them you can store more information about the corresponding sequence, including references and a so called Features table that contains more in depth information about the sequence

genbank files II [4]

- The first line of each genbank file is the LOCUS field (red circle) and it contains information about:
  - the locus name (blue circle)
  - the sequence length in bp (base pairs) or aa (amino acids) (green circle)
  - the molecule type (purple circle)
  - the genbank division (orange circle)
  - the modification date (dark red circle)



genbank files III [4]

- After the LOCUS field some more fields containing general information about the sequence follow:
  - The DEFINITION field contains a brief description of the sequence

```
DEFINITION  Hepatitis C virus NS5b gene.
```

  - The ACCESSION field contains the unique identifier for the sequence

```
ACCESSION   Z97730
```

  - The VERSION field contains the unique identifier of the sequence with the added version Nr in point notation, e.g. .1 refers to the first version

```
VERSION     Z97730.1
```

  - The KEYWORD field contains words or phrases describing the sequence

```
KEYWORDS    NS5b gene; RNA-dependent RNA-polymerase.
```

genbank files IV [4]

- After the LOCUS field some more fields containing general information about the sequence follow:
  - The SOURCE field contains an abbreviated form of the organism name

```
SOURCE      Hepacivirus C
```

  - The ORGANISM field contains the scientific name of the organism and in the followoing lines the lineage based on the phylogenetic classification scheme of the NCBI

```
  ORGANISM  Hepacivirus C
            Viruses; Riboviria; Orthornavirae; Kitrinoviricota; Flasuviricetes;
            Amarillovirales; Flaviviridae; Hepacivirus.
```

genbank files V [4]

- After these general information genbank files contain information about all publications that discuss the sequence in one or more REFERENCE fields
- The publications are sorted by publication data, with the oldest being displayed as first reference

```
REFERENCE   1
  AUTHORS   Lohmann,V., Korner,F., Herian,U. and Bartenschlager,R.
  TITLE     Biochemical properties of hepatitis C virus NS5B RNA-dependent RNA
            polymerase and identification of amino acid sequence motifs
            essential for enzymatic activity
  JOURNAL   J. Virol. 71 (11), 8416-8428 (1997)
   PUBMED   9343198
```

genbank files VI [4]

- The REFERENCE fields are followed by the FEATURES table that can contain lots of different data, depending on the nature of the sequence
- Mandatory is the SOURCE field that summarizes the general information of fields like ORGANISM:

```
FEATURES             Location/Qualifiers
     source          1..1776
                     /organism="Hepacivirus C„
                     /mol_type="genomic RNA„
                     /isolate="RB01„
                     /db_xref="taxon:11103"
```

genbank files VII [4]

- The FEATURES table can also include information about regions of biological interest in form of GENE, PROTEIN or REGION fields:

```
gene            <1..1776
                /gene="NS5b„
Protein         1..467
                /name="tms2„
Region          1..464
                /region_name="PRK07488„
                /note="indole acetimide hydrolase; Validated„
                /db_xref="CDD:236030„
```

- These fields contain spans, e.g. 1..467 to specify which part of the sequence is meant with the corresponding field

genbank files VIII [4]

- Furthermore the FEATURES table can also include CDS fields, which contain information about coding sequences, that means regions that correspond to specific proteins:

```
CDS             <1..1776
                ...
                /translation="SMSYTWTGALITPCAAEESKLPINALSNSLLRHHNMVYATTSRS
                ASLRQKKVTFDRLQVLDDHYRDVLKEMKAKASTVKAKLLSVEEACKLTPPHSAKSKFG
                ...
                ARLLSQGGRAATCGKYLFNWAVRTKLKLTPIPAASQLDLSSWFVAGYSGGDIYHSLSR
                ARPRWFMWCLLLLSVGVGIYLLPNR„
```

- Aside many other information, the CDS field includes a translation of the corresponding nucleotide sequence, however this translation is often conceptual

genbank files IX [4]

- genbank files end with the ORIGIN field that contains the whole sequence:

```
ORIGIN
        1 mvaitslaqs lehlkrkdys clelvetlia rceaakslna llatdwdglr rsakkidrhg
       61 nagvglcgip lcfkaniatg vfptsaatpa linhlpkips rvaerlfsag alpgasgnmh
      121 elsfgitsnn yatgavrnpw npdlipggss ggvaaavasr lmlggigtdt gasvrlpaal
      181 cgvvgfrptl grypgdriip vsptrdtpgi iaqcvadvvi ldriisgtpe rippvplkgl
      241 riglpttyfy ddldadvala aettirllan kgvtfveani phldelnkga sfpvalyefp
      301 halkqylddf vktvsfsdvi kgirspdvan ianaqidghq iskaeyelar hsfrprlqat
      361 yrnyfklnrl dailfptapl varpigqdss vihngtmldt fkiyvrnvdp ssnaglpgls
      421 ipvcltpdrl pvgmeidgla dsdqrllaig galeeaigfr yfaglpn
//
```

- Each file ends with the // field which indicates the end of one entry

genbank files X

- While genbank files mostly consist of information about one single sequence, they are able to contain multiple entries at once as well
- Each entry will start with its own LOCUS field and end with the // field, that means the // field functions as end character for files and as separator for multiple entries in the same file

genbank files XI

- We can read in genbank files again with the help of the SeqIO submodule of biopython and its parse() function:

```
genbank1 = SeqIO.parse("455373.gb", "genbank")
```

- The variable genbank1 is a GenBankIterator object which contains one sequence object for each sequence in our genbank file
- That means we need to iterate over it with the help of a for loop for example to get access to the different sequence objects:

```
for record in genbank1:
```

genbank files XII

- Each sequence object generate by the parse() function of SeqIO has several different attributes and the four most interesting ones are:

```
record.id
record.description
record.seq
record.features
```

- While the first three contain single strings with the corresponding information, the last one contains a list with several SeqFeature objects, one for each entry in the FEATURES table of the corresponding sequence

clustalW files I [5]

- Many if not all multiple sequence alignment tools support the clustalW format for their outputs
- They often have the extension .aln but .clustal is also possible
- These files contain the result of a multiple sequence Alignment and follow very specific rules
- Each file must start with the words "CLUSTAL" or "CLUSTALW“ and other information in the first line is ignored:

```
CLUSTAL O(1.2.4) multiple sequence alignment
```

- This is followed by one or more empty lines

clustalW files II [5]

- Afterwards they contain one or more blocks of sequence data
- Each block consists of multiple lines with one line representing one sequence of your multiple sequence alignment
- Each line consists of:
  - the sequence name
  - white space(s)
  - up to 60 sequence symbols
  - optional - white space followed by a cumulative count of residues for the sequences

```
XP_031236805.1      MNGTEGPNFYVPFSNITGVVRSPFEQPQYYLAEPWQFSMLAAYMFLLIVLGFPINFLTLY
XP_028609402.1      MNGTEGPNFYVPFSNVTGVVRSPFEQPQYYLAEPWQFSMLAAYMFLLIVLGFPINFLTLY
```

clustalW files III [5]

- The sequence lines of a block are than followed by:
  - A line showing the degree of conservation for the columns of the alignment in this block
  - One or more empty lines

```
NP_663358.1         ATTQKAEKEVTRMVIIMVIFFLICWLPYASVAFYIFTHQGSNFGPIFMTLPAFFAKSSSI
AAA39861.1          ATTQKAEKEVTRMVIIMVIFFLICWLPYASVAFYIFTHQGSNFGPIFMTLPAFFAKSSSI

                    ****************:***************.**:**************************
```

clustalW files IV [5]

- The characters used to represent the degree of conservation are:

  * -- all residues or nucleotides in that column are identical (red rectangle)

  : -- conserved substitutions have been observed (green rectangle)

  . -- semi-conserved substitutions have been observed (blue rectangle)

  -- no match (orange rectangle)

clustalW files V [6]

- The degree of conservation clustalW files display is calculated according to the descriptions shown in the table below
- PAM 250 refers to a so called substitution matrix, that contains numerical values for mutation probabilities of amino acids, we will talk more about them later when we look at Alignments

Symbol	Definition	Meaning
*	asterisk	positions that have a single and fully conserved residue
:	colon	conservation between groups of strongly similar properties with a score greater than .5 on the PAM 250 matrix
.	period	conservation between groups of weakly similar properties with a score less than or equal to .5 on the PAM 250 matrix

clustalW files VI

- Since clustalW files contain sequence alignments we won‘t discuss reading them in here in detail
- Of course the SeqIO parser from biopython is again able to read in these files, so that we could work with them in our python scripts
- An alternative is the AlignIO submodule of biopython and you can look at an example for this module in the script „18_Files-Clustal.py“

pdb files I

- We know that analyzing the function of proteins is only possible in combination with knowledge about their three-dimensional structure
- Therefore the RCSB-PDB as part of the world wide PDB (wwPDB) created a file format that allows the storage of three-dimensional structural data in form of atom coordinates
- This is done by using so called pdb-files and nowadays nearly all coordinate data is saved either in pdb format or the newer mmCIF format, which still follows some of the same rules
- Files in the pdb format have the extension .pdb

pdb files II [7]

- The general structure of pdb files follows very strict rules
- Each line, which contain a maximum of 80 columns, that means 80 characters, in a pdb file represents one record and has to be self-identifying
- The first six columns of a line contain the record name (red circle), which must be one of the predefined record names of the pdb format
- The record name is than followed by a space and the remaining information / data that belongs to this record:

```
REMARK 500 GEOMETRY AND STEREOCHEMISTRY
REMARK 500 SUBTOPIC: COVALENT BOND ANGLES
```

pdb files III [7]

- In general pdb files contain multiple sections:
  1. Title section
  2. Primary structure section
  3. Heterogen section
  4. Secondary structure section
  5. Connectivity Annotation section
  6. Miscellaneous Features section
  7. Crystallographic and Coordinate Transformation section
  8. Coordinate section
  9. Connectivity section
  10. Bookkeeping section

pdb files IV [7]

- On the next slides we will briefly discuss each of the different sections except for the coordinate and secondary structure section where we will go a bit more into detail, since they contain often times the most interesting data when working with these files
- We will see what we can do with these files later on in our part about structural bioinformatics, where we will discuss how we can visualize these structural data and how we can run calculations with this data in Python

pdb files V [7]

- The **Title section** of a pdb file contains information about the experiment and the biological macromolecules present in the file
- It can contain up to 16 different record types
- The first record is always the HEADER, which contains a classification of the molecule(s) (red bracket), the deposition date (green bracket) and the PDB identifier (blue bracket):

pdb files VI [7]

- Other noteworthy records in the title section are:
  - TITLE, contains the title for the experiment or analysis
  - COMPND, describes the macromolecular contents of the file
  - SOURCE, specifies the biological and/or chemical source of each biological molecule in the file
  - AUTHOR, contains the names of the people responsible for the contents of the file
  - JRNL, contains the primary literature citation that describes the experiment
  - REMARK, presents experimental details, annotations, comments and information not included in other record

pdb files VII [7]

- The REMARK section is further devided into many subsections, each marked by a specific number
- While the information we can read out of these records can be very interesting when working in structure biology, we won‘t go into detail about most of them
- For our purposes it‘s enough to know about REMARK 465 and REMARK 470

pdb files VIII [7]

- REMARK 465 lists the residues, refering to either amino acids or bases in our macromolecules, that are present in the sequence which is given in the same file but are completely absent from the coordinate section
- The main part of this section is a table containing:
  - the residue name, e.g. SER (red)
  - the chain identifier, e.g. A, that denotes one of the macromolecules present in the file (green)
  - sequence number, e.g. 11, the position of the residue in the sequence (blue)

```
REMARK 465   M RES C SSSEQI
REMARK 465     SER A   11
REMARK 465     ALA A   12
REMARK 465     PHE A   13
```

pdb files IX [7]

- REMARK 470 contains information about the non-hydrogen atoms of standard residues which are missing from the coordinates
- The main part is again a table containing:
  - the residue name (red)
  - the chain identifier (green)
  - the sequence number (blue)
  - the names of the missing atoms (orange)

```
REMARK 470   M RES C SSEQI   ATOMS
REMARK 470     MET A   46     CG    SD    CE
REMARK 470     VAL A   70     CA    C     O     CB    CG1   CG2
```

pdb files X [7]

- The **primary structure section** of pdb files is reserved for the sequence(s) of the macromolecule(s) present in the file
- The sequences are saved under the SEQRES record
- Each record contains:
  - the serial number (red)
  - the chain identifier (green)
  - the number of residues in the chain (blue)
  - the residue names of up to 13 consecutive residues (orange)

```
SEQRES   1  A  366  SER ALA PHE ASP GLN ALA ALA ARG SER ARG GLY HIS SER
SEQRES   2  A  366  ASN ARG ARG THR ALA LEU ARG PRO ARG ARG GLN GLN GLU
SEQRES   3  A  366  ALA THR GLU VAL ARG PRO GLU GLN LYS MET PRO THR LEU
```

pdb files XI [7]

- The **Heterogen Section** contains the complete description of non-standard residues in the file
- This can refer to ligands, co-factors like metal ions or to inhibitors of the macromolecule
- Here we can also find information about modified amino acids, e.g. glycosylated amino acids

pdb files XII [7]

- The **secondary structure section** describes helices and sheets found in protein and polypeptide structures (more about this during the structure bioinformatics part)
- The **connectivity annotation section** allows the depositors to specify the existence and location of disulfide bonds and other linkages
- The **miscellaneous features section** may describe properties in the molecule such as environments surrounding a non-standard residue or the assembly of an active site
- **The Crystallographic and Coordinate Transformation Section** describes the geometry of the crystallographic experiment

pdb files XIII [7]

- The **coordinate section** is the most important section of the whole file, since here we can find the exact three-dimensional coordinates of all atoms present in the structure
- The whole section is built in a tabular way to contain all necessary information for each atom
- One important thing to keep in mind than looking at this tabular data (next slide) is that the different “columns” depend on the text columns, that means their position in the line and NOT spaces or things like this
- In total this section contains the records MODEL, ATOM, ANISOU, TER, HETATM and ENDMDL

pdb files XIV [7]

- Down below you can see a part of the coordinate section of a pdb file with 6 ATOM records for a cysteine:

```
ATOM    841  N   CYS A 171      38.741  77.834  55.215  1.00 11.75           N
ATOM    842  CA  CYS A 171      39.671  77.832  56.336  1.00 12.29           C
ATOM    843  C   CYS A 171      40.108  79.166  56.902  1.00 12.21           C
ATOM    844  O   CYS A 171      39.952  79.408  58.087  1.00 10.41           O
ATOM    845  CB  CYS A 171      40.858  76.933  56.017  1.00 11.97           C
ATOM    846  SG  CYS A 171      40.345  75.276  55.592  1.00 12.82           S
```

record name columns 1-6

atom number columns 7-11

atom name columns 13-16

residue name columns 18-20

chain identifier column 22

residue number columns 23-26

x-coordinate columns 31-38

y-coordinate columns 39-46

z-coordinate columns 47-54

element symbol columns 77-78

pdb files XV [7]

- As you could see each atom in an amino acid has a very specific name
- Non-hydrogen atoms have names with a maximum of 3 letters, while hydrogen atom names can contain up to 4 letters
- The exact names for each atom in amino acids, nucleotides and all other kinds of molecules are defined by the wwPDB conglomerate [8]

pdb files XVI [7]

- ANISOU and HETATM records follow similar rules as we could saw for the ATOM records, since they also specify the three-dimensional coordinates
- TER records on the other hand indicate the end of a list of ATOM/HETATM records for one macromolecular chain
- They simply contain:
  - The atom number in columns 7-11 (red)
  - The residue name in columns 18-20 (green)
  - The chain identifier in column 22 (blue)
  - The residue number in columns 23-26 (orange)

pdb files XVII [7]

- The **connectivity section** provides information about different kind of bonds in the structure
- This can include Disulfide bonds, bonds inside ligands and so on
- Only bonds that are not specified in the standard residue connectivity table [8] are noted here

pdb files XVIII [7]

- Last but not least the bookkeeping section provides some final information about the file itself
- It contains the MASTER record which lists the number of lines in the coordinate entry or file for selected record types
- It also contains the END record which marks the end of the PDB file

```
MASTER      490    0    4   32   14    0   13    9 4933    2   44   58
END
```

Literature

1. https://docs.python.org/3/library/re.html?highlight=re#module-re (Last edited: 18.01.2023, Last visited: 18.01.2023)
2. https://biopython.org/wiki/SeqIO (Last visited: 04.05.2023)
3. https://fastaparser.readthedocs.io/en/latest/ (Last visited: 04.05.2023)
4. https://www.ncbi.nlm.nih.gov/Sitemap/samplerecord.html (Last edited: 23.10.2006, Last visited: 04.05.2023)
5. https://meme-suite.org/meme/doc/clustalw-format.html (Last visited: 04.05.2023)
6. https://en.wikipedia.org/wiki/Clustal (Last edited: 12.04.2023, Last visited: 04.05.2023)
7. http://www.wwpdb.org/documentation/file-format-content/format33/v3.3.html (Last edited: 21.11.2012, Last visited: 04.05.2023)
8. https://files.wwpdb.org/pub/pdb/data/monomers/ (Last edited: 28.04.2023, Last visited: 04.05.2023)
9. https://en.wikipedia.org/wiki/FASTQ_format (Last edited: 09.06.2023, Last visited: 04.07.2023)